SYSTEM DESIGN REPORT

Grace Community Church Platform

A technical walkthrough of the platform: the multi-tenant architecture, the Members section workflow, and the data model that connects them. A shared reference for the team.

Prepared for the Ministry Team & Stakeholders

Version 2.0 · October 2026

Table of Contents

1. System Architecture 2

2. Members Section Workflow 4

3. Data Model 6

4. Optimization Notes for Later 8

1. System Architecture

The platform is a single Next.js 16 fullstack monolith: one codebase serves the React 19 admin single-page app, the public member portal, and fifteen groups of API route handlers. Data access runs through Prisma ORM, currently pointed at a SQLite database for local development, with sharp processing member photos at upload time. Everything an administrator touches — members, households, groups, events, attendance, media, series, submissions, volunteers, tags, reminders, statistics and audit logs — flows through the same request pipeline, and that uniformity is what keeps the security posture consistent as features are added.

Multi-tenancy follows a shared-database, shared-schema pattern: every table carries an **orgId** tenant column and every query is scoped by it, with a foreign-key re-check before writes so one organisation can never touch another's rows. Sessions are HMAC-SHA256 signed cookies (**gc_session**, seven-day expiry) verified in edge middleware using the Web Crypto API before any route handler executes; the middleware keeps an explicit allow-list for the login endpoint, the session check, and the public portal routes. If SESSION_SECRET is missing at boot, the server refuses to start rather than running unsecured.

Every write request traverses the same six-step guard pipeline shown in Figure 1: rate limiting, session and role checks, schema validation, organisation scoping, the transactional handler, and an audit write. Portal submissions record audit rows with a null administrator identity, keeping the trail honest about who performed each change. A central error handler maps Prisma faults — unique-constraint and not-found violations — onto clean HTTP responses so internal details never leak to clients.

15	60/min	3	orgId
API resource groups	write rate limit per IP	RBAC roles (admin / editor / viewer)	tenant column on every table

Table 1 summarises the pipeline in the order requests actually experience it, together with the response each guard returns when it rejects a request. The ordering is deliberate: cheap checks run first so abusive traffic is shed before it reaches the database, and the audit row is written only after a transaction commits, so the log never records changes that failed.

#	Guard step	What it verifies	On failure
1	Rate limit	60 writes per minute per IP (sliding window)	429
2	Session + RBAC	Valid HMAC cookie; role meets the endpoint's requirement	401 / 403
3	Zod validation	Request body matches the resource schema	400
4	Org scoping	Target rows belong to the caller's organisation	404
5	Handler + Prisma	All-or-nothing transactional write	500

#	Guard step	What it verifies	On failure
6	Audit write	One AuditLog row per change (non-blocking)	500

Table 1 — The shared guard pipeline executed by every write endpoint.

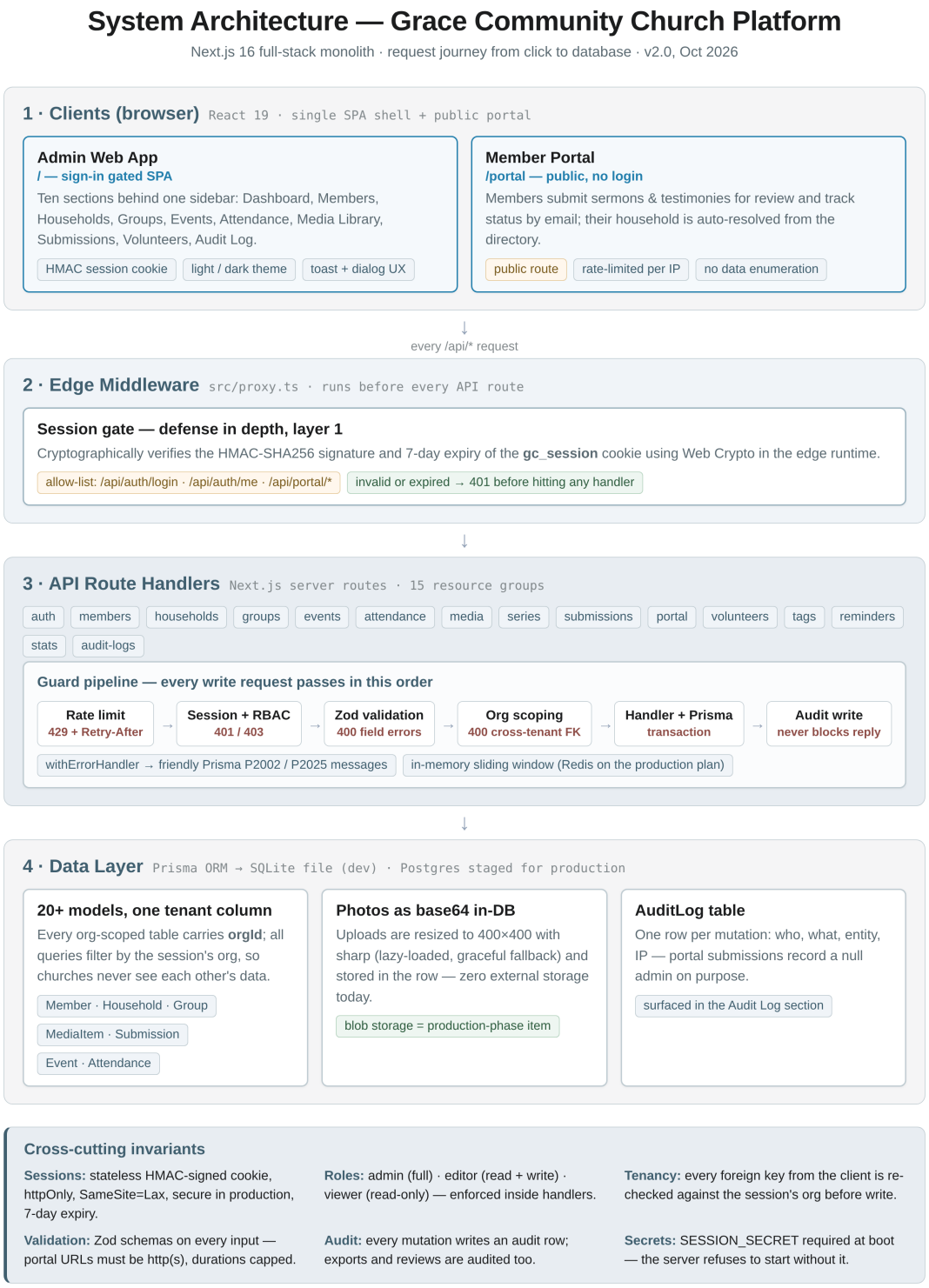


Figure 1 — System architecture: client apps, edge middleware, the API guard pipeline, and the data layer, with the cross-cutting invariants every request inherits.

Reading the figure top to bottom mirrors a real request: clients issue calls, the edge layer authenticates the session, route handlers run the guard pipeline against the Prisma data layer, and audit rows land beside the business tables. The invariants grid at the bottom — tenant scoping, auditability, validated input, rate-limited writes — applies to every arrow above it, which is why new modules inherit the same guarantees without extra code.

2. Members Section Workflow

The Members section is the busiest part of the admin app, so it is the clearest lens on how any module behaves end to end. The interface exposes six actions: browsing and searching the roster, adding a member, editing an existing record, uploading a photo, bulk import and export via CSV, and deletion. Each action calls the same guarded API every other module uses — there is no separate path that bypasses the pipeline described in Chapter 1.

Tracing one happy path makes the flow concrete. When an editor saves a member edit, the SPA sends a JSON payload to **/api/members/[id]**. The request passes edge session verification, then the rate limiter, then the role check (editor or admin for writes), then Zod validation of the field values, and finally the organisation re-check that the target member belongs to the caller's church. Only then does the handler open its Prisma transaction; on success an audit row records who changed what, and the freshly written record returns to the UI, which updates the roster in place.

The error paths are equally intentional, and Figure 2 draws each one. A 400 returns field-level validation detail; 401 means the session is missing or expired; 403 means the role is too low for the action. A 404 is returned when a record exists but belongs to another organisation — deliberately indistinguishable from a missing record so the API leaks no existence information across tenants. A 429 answers over-eager clients, and 500 is reserved for genuine faults. CSV import runs through a dry-run validation endpoint first, so spreadsheet mistakes are caught before any row is written, and deletions cascade explicitly — a member's submissions are removed with them.

Members Section — End-to-End Workflow

From an admin's click to the database and back — the same guard chain protects every write

Phase 1 · Admin UI — Members section `src/components/sections/MembersSection.tsx`

1. **Browse the directory** — table loads via `GET /api/members`; debounced search (300 ms) plus status, group and household filters re-query the API.
2. **Add member** — dialog form with client-side Zod rules (gender + maritalStatus required, amber DOB warning); household and groups can be created inline on the fly.
3. **Edit member** — detail dialog pulls the full profile (household, groups, tags, attendance, submissions) then patches only the changed fields.
4. **Upload photo** — multipart `POST /api/members/[id]/photo`; the image is cropped square in the browser flow and stored with the member.
5. **Import / export CSV** — a three-step wizard (upload → preview → commit) for bulk adds; one click exports the filtered list as CSV.
6. **Delete member** — confirm dialog, then `DELETE /api/members/[id]`; memberships, tags and attendance cascade.

↓ every action funnels through the same server-side chain

Phase 2 · API guard pipeline run in order, stop at first failure

7. **Rate limit** — 60 write requests per minute per IP (login and portal have tighter buckets); excess gets `429` with a Retry-After header.
8. **Session + RBAC** — the HMAC cookie is verified again inside the handler and the role is checked: only **admin** and **editor** may write.
9. **Zod validation** — `createMemberSchema` / `updateMemberSchema` parse the payload; bad input returns `400` with per-field messages.
10. **Org scoping + uniqueness** — duplicate email inside the org is rejected; every household / group id is proven to belong to the admin's church before it is referenced.

↓ all checks passed

Phase 3 · Data layer & response

11. **Prisma transaction** — the member row and its group memberships are written atomically to SQLite, always stamped with `orgId`.
12. **Audit trail** — `member.create` / `member.update` / `member.delete` is recorded with the admin, action, entity and IP.
13. **Response** — `201` / `200` JSON returns to the UI, the list refetches and a toast confirms; CSV exports stream a file attachment instead.

Error paths — what the admin sees when a step fails

- `400` — validation failed, duplicate email, or a foreign key from another church
- `401` — no cookie or expired session → back to the sign-in screen
- `403` — viewer role attempted a write
- `404` — member exists but outside your organization (never leaked)
- `429` — too many requests; UI shows the retry countdown
- `500` — friendly message; full detail only in the server log

Figure 2 — The Members section end to end: six UI actions, the four-step API guard, the transactional data layer, and the six error paths that keep behaviour predictable.

The diagram's three phases correspond to the three columns of concern the team optimises independently: the UI phase is pure presentation and can be redesigned freely; the guard phase is shared infrastructure that should change rarely and deliberately; the data phase defines durability and traceability guarantees. Keeping the boundaries clean is what allows member-management features to iterate quickly without re-opening security questions each time.

3. Data Model

The schema is organised around one hub: **Member**. Households group people who share an address (with an optional head-of-household pointer back to a member); groups connect through a Membership join table whose member-plus-group pair is unique, carrying a role of leader, member or volunteer; attendance links a member to an event with a recorded status. Tags attach many-to-many through three dedicated join tables — MemberTag, MediaTag and EventTag — so one label vocabulary serves people, media and events without cross-contamination.

The media pipeline adds two cooperating tables. A **Submission** is a portal draft: it captures the submitting member, an optional note, a reviewer, and a status of pending, approved or rejected, and it links one-to-one with the MediaItem it proposes. A **MediaItem** is the library asset itself — sermon, video, document or audio — with a status of draft, published or archived, an optional SermonSeries, and a speakerId pointing at Member. That speaker field is what surfaces a person's name on Media Library cards; it was recently hardened so portal submissions set it at creation time and approval backfills it whenever it is empty.

Relationship	Cardinality	Notes
Household → Member	N : 1	People share a household; headMemberId is optional
Membership (Member ↔ Group)	M : N	Unique pair; role = leader / member / volunteer
Attendance (Member ↔ Event)	M : N	Per-event record with status and timestamp
Tag ↔ Member / Media / Event	M : N	Three join tables, one tag vocabulary
Submission → MediaItem	1 : 1	Portal draft reviewed into a library asset
MediaItem → SermonSeries	N : 1	Deleting a series detaches media, never deletes it
MediaItem.speakerId → Member	N : 1	Name shown on Media Library cards

Table 2 — Key relations around the Member hub.

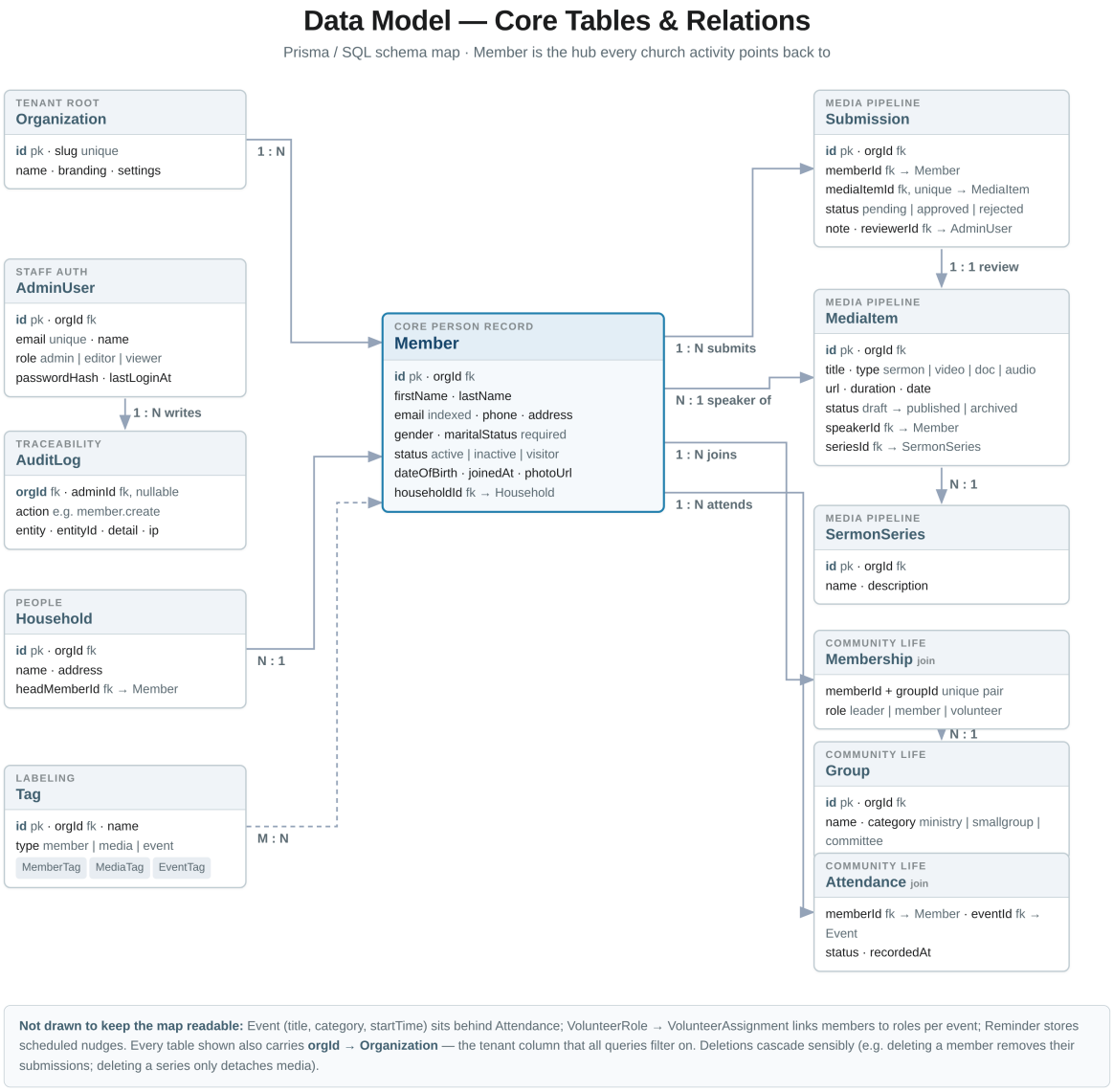


Figure 3 — Core tables and relations. Every table also carries the orgId tenant column (omitted per-card for readability); Event, VolunteerRole and Reminder sit behind Attendance and the join tables shown.

Two conventions hold across the whole schema. First, every table carries orgId, so tenant scoping is a property of the data rather than of any query. Second, deletions cascade deliberately: removing a member removes their submissions and attendance rows, while removing a series or a tag only detaches references. The note strip at the bottom of Figure 3 lists the entities left out of the map for readability — Event details, the volunteer role and assignment tables, and reminders — none of which introduce patterns absent from the diagram.

4. Optimization Notes for Later

The current design makes a consistent trade: it leans on single-process conveniences to keep local development simple, and every one of those trade-offs was agreed to be deferred rather than fixed prematurely. None of them affects day-to-day use on a single instance, and each is isolated behind the guard pipeline, so none will force a rewrite when it is revisited. They are recorded here so the team can plan for them deliberately.

Three items form the main list. First, the write rate limiter lives in process memory — a sliding window that is correct for a single server but resets on restart and cannot be shared across instances; moving it to Redis is the standard next step once the platform runs on more than one node. Second, member photos are processed by sharp and stored on local disk under the public uploads directory, which should move to blob or object storage for backups and horizontal scale. Third, SQLite serves local development well, but PostgreSQL is the production target; because all access goes through Prisma and the schema avoids SQLite-specific tricks, the switch is a provider change plus a migration replay rather than a code rewrite.

Two smaller observations belong on the same list. The audit log grows without a retention policy and will eventually want archiving or pruning. And sessions are fixed seven-day HMAC cookies without a revocation list, which is acceptable today but worth revisiting if staff turnover or shared-device usage becomes a factor. Per the standing agreement, all of these remain intentionally out of scope for now — the right moment to pick them up is immediately before a multi-instance production deployment, when their value becomes concrete.